

Laporan Praktikum 4 Kontrol Cerdas

Nama : Arjuna Kurniawan Kusuma Putra

NIM : 224308004

Kelas : TKA-6A

Akun Github (Tautan) : <https://github.com/kakjuna>

Student Lab Assistant : Rizky Putri Ramadhani (214308092)

1. Judul Percobaan

Melatih dan Menguji Agen Reinforcement Learning Untuk Mengontrol Lingkungan Secara Otonom

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

1. Mahasiswa dapat memahami Konsep dasar dari Reinforcement Learning (RL) dalam sistem kendali.
2. Mahasiswa dapat mengetahui cara mengimplementasikan RL menggunakan algoritma Deep Q-Network (DQN).
3. Mahasiswa dapat mengetahui bagaimana melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.

3. Landasan Teori

- Reinforcement Learning (RL) merupakan salah satu cabang dari Machine Learning yang berbeda dari supervised learning dan unsupervised learning. Metode ini berfokus pada pengembangan agen cerdas yang dapat belajar dari interaksi dengan lingkungan (environment). Dalam RL, agen bertujuan mencapai suatu target dengan menerima feedback berupa reward atau penalty atas setiap tindakan yang dilakukan. Agen kemudian belajar untuk memaksimalkan akumulasi reward atau meminimalkan penalty guna mencapai tujuan dengan lebih optimal.
- Environment merupakan elemen penting dalam pengembangan Reinforcement Learning (RL). Environment berperan dalam memberikan observasi dari lingkungan simulasi kepada agen RL, menerima dan mengeksekusi aksi yang dilakukan oleh agen, serta mengevaluasi aksi tersebut dengan memberikan reward feedback kepada agen. Dalam pengembangan algoritma RL, keberadaan environment sangat krusial, karena tidak memungkinkan untuk langsung menguji algoritma dalam lingkungan fisik. Implementasi langsung pada sistem nyata, seperti robot, menghadapi berbagai tantangan, termasuk biaya tinggi, banyaknya variabel tak terkontrol yang dapat mempengaruhi hasil, serta kendala fisik yang dapat memperlambat proses pelatihan agen. Oleh karena itu, simulasi environment digunakan untuk menguji dan mengoptimalkan algoritma sebelum diterapkan di dunia nyata.
- OpenAI Gym adalah pustaka open-source yang dikembangkan oleh tim OpenAI untuk mendukung pengembangan algoritma Reinforcement Learning (RL). OpenAI Gym ditulis dalam bahasa pemrograman Python, yang banyak digunakan oleh peneliti machine learning dan data science dalam mengembangkan serta mengimplementasikan berbagai algoritma. Gym dirancang untuk

mempermudah pengembangan algoritma RL, mengatasi kendala utama yang sering dihadapi oleh peneliti, yaitu ketiadaan standar dalam pengujian dan perbandingan algoritma. Tanpa environment standar, sulit untuk menguji dan mereproduksi hasil penelitian yang dilakukan oleh peneliti lain. Selain itu, membangun environment sendiri akan memakan waktu dan sumber daya yang signifikan, sehingga OpenAI Gym menyediakan berbagai environment siap pakai untuk mempercepat proses penelitian.

- Python adalah bahasa pemrograman yang populer. Python sering dimanfaatkan dalam pengembangan web, perangkat lunak, penelitian, dan system scripting. Python dapat digunakan untuk 39 menangani data besar dan melakukan operasi matematika yang kompleks. Python bekerja di berbagai platform seperti Windows, Mac, Linux, Raspberry Pi, dan lain-lain. Python dirancang untuk mudah dibaca, yaitu memiliki sintaks yang sederhana dan menggunakan bahasa inggris.
- NumPy (Numerical Python) merupakan library Python yang fokus pada scientific computing. NumPy memiliki kemampuan untuk membentuk objek N-dimensional array, yang mirip dengan list pada Python. Struktur data NumPy lebih membutuhkan ukuran yang lebih kecil dibandingkan dengan list lainnya tetapi mempunyai performa yang lebih cepat
- TensorFlow adalah salah satu library software yang dikembangkan oleh Google Brain dalam organisasi penelitian Google Smart Machine. Teknologi ini dirancang untuk pembuatan model machine learning serta penelitian jaringan saraf tiruan. TensorFlow menggabungkan aljabar komputasi dengan teknik optimasi kompilasi, sehingga dapat mempercepat perhitungan ekspresi matematis yang kompleks, terutama dalam mengatasi masalah efisiensi waktu komputasi. Berikut beberapa fitur utama TensorFlow (Abadi et al., 2016):
 1. Memungkinkan definisi, optimasi, dan perhitungan efisien terhadap ekspresi matematis yang melibatkan array multidimensi (tensors).
 2. Mendukung pemrograman berbasis kecerdasan buatan (AI) dan machine learning.
 3. Memanfaatkan GPU secara optimal, dengan otomatisasi manajemen memori dan data. TensorFlow dapat menjalankan kode yang sama di CPU maupun GPU, serta menentukan secara otomatis bagian perhitungan yang perlu diproses di GPU.
 4. Skalabilitas tinggi, mampu menangani komputasi dalam skala besar di berbagai mesin dan dataset yang besar.

TensorFlow memiliki berbagai fitur yang mendukung training, testing, dan deployment model machine learning dengan lebih efisien.

4. Analisis dan Diskusi

Analisis:

Pada minggu ke-4 percobaan ini, saya melakukan pelatihan dan pengujian agen Reinforcement Learning (RL) agar dapat mengontrol lingkungan secara otomatis. Saya menggunakan algoritma Deep Q-Network (DQN) sebagai metode RL dan memanfaatkan TensorFlow untuk membangun serta menjalankan modelnya. Proses ini terdiri dari dua tahap, yaitu pelatihan dan pengujian agen RL. Program pertama digunakan untuk melatih agen, sedangkan program kedua digunakan untuk menguji agen setelah dilatih. Saat kedua program dijalankan, dihasilkan data latih dan data uji. Dari hasil percobaan, agen belum mampu mengontrol CartPole dengan baik karena skor pelatihan tidak mengalami peningkatan yang signifikan, sehingga skor uji tetap rendah.

Namun, performa agen dapat ditingkatkan dengan menyesuaikan beberapa parameter. Jika nilai gamma tinggi, agen akan lebih fokus pada reward jangka panjang, yang dapat meningkatkan stabilitas pembelajaran. Sebaliknya, jika nilai gamma rendah, agen akan lebih fokus pada reward instan, yang dapat mempercepat proses pembelajaran. Untuk parameter epsilon, jika nilainya tinggi, agen akan lebih sering mencoba aksi secara acak (eksplorasi), yang sangat berguna di awal pelatihan. Sebaliknya, jika nilainya rendah, agen akan lebih sering memilih aksi terbaik (eksploitasi), yang lebih cocok digunakan pada akhir pelatihan. Sedangkan pada learning rate, jika nilainya tinggi, agen dapat belajar lebih cepat, tetapi bisa menjadi tidak stabil dan rentan terhadap overfitting. Jika learning rate rendah, agen akan belajar lebih lambat, tetapi lebih stabil dan memiliki performa yang lebih baik dalam jangka panjang. Dengan menyesuaikan parameter tersebut, diharapkan agen dapat belajar lebih baik

dan mampu mengontrol lingkungan dengan lebih optimal.

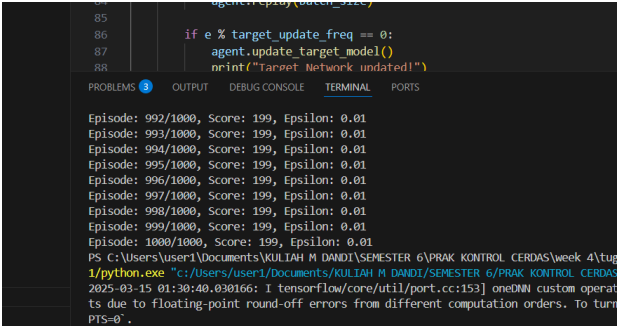
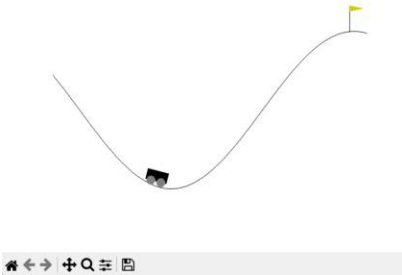
Diskusi:

1. Perbedaan utama antara RL dan Supervised Learning dalam sistem kendali dapat dilihat dari cara belajar dan sumber data. Pada RL dalam melakukan pembelajaran melalui trial dan error dengan mengeksplorasi lingkungan dan menerima reward dan RL tidak memerlukan dataset berlabel karena dapat mengumpulkan data sendiri dengan berinteraksi dengan lingkungan. Sedangkan Supervised Learning dalam melakukan pembelajaran melalui data berlabel yang telah diberikan sebelumnya sehingga membutuhkan dataset berlabel besar untuk melatih model dengan akurasi tinggi.
2. Cara mengoptimalkan keseimbangan antara eksplorasi dan eksploitasi yaitu, pada awal pelatihan gunakan lebih banyak eksplorasi dengan menggunakan ϵ -Greedy, Boltzmann, atau UCB untuk mengeksplorasi lingkungan. Ketika sudah berjalan seiring waktu gunakan eksploitasi bisa dengan menggunakan ϵ -decay atau Dueling DQN untuk menemukan strategi terbaik. Jika kondisi masalah dengan reward jarang gunakan curiosity-driven exploration.
3. Aplikasi lain dari RL dalam sistem kendali nyata yang dapat diimplementasikan yaitu:
 - Kendali kendaraan otonom (Self-Driving Cars & Autonomous Rail).
 - Kendali Robotik dan otomasi industry.
 - Kendali pergerakan kereta api.

5. Assignment

Pada assignment, program latih dimodifikasi dengan menambahkan target network setiap 1000 episode agar dapat meningkatkan stabilitas pelatihan. Selain itu pada program latih dan uji diubah environmentnya yang awalnya menggunakan CartPole-v1 menjadi MountainCar-v0. Sehingga ketika kedua program dijalankan menghasilkan nilai score pada data latih dan data uji yang stabil disetiap episode sebesar 199, sedangkan untuk nilai epsilonya menurun. Namun dari data yang didapat dapat disimpulkan bahwa agen masih belum mampu menyelesaikan tugasnya. Hal ini terjadi karena MountainCar-v0 membutuhkan pelatihan yang lebih lama dengan episode yang lebih tinggi.

6. Data dan Output Hasil Pengamatan

NO	Variabel	Hasil Pengamatan
1	Proses pelatihan data	
2	Hasil	

7. Kesimpulan

Dari percobaan yang telah dilakukan dapat disimpulkan bahwa:

1. Percobaan ini dilakukan dengan menggunakan TensorFlow untuk membangun model Deep Q-Network (DQN) yang nantinya digunakan untuk mengimplementasikan agen RL dan menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
2. Pada percobaan ini terdiri dari 2 tahapan, yaitu tahap pelatihan agen yang digunakan untuk melatih agen dan tahap pengujian agen yang digunakan untuk menguji agen RL yang sebelumnya sudah dilatih.
3. Hasil percobaan menunjukkan bahwa agen belum mampu mengontrol CartPole dengan baik, yang ditunjukkan dengan skor pelatihan yang tidak meningkat secara signifikan serta skor uji yang rendah. Hal ini disebabkan oleh pemilihan parameter yang kurang optimal, seperti gamma, epsilon, dan learning rate.

8. Saran

Berdasarkan hasil percobaan yang telah dilakukan, terdapat beberapa saran yang penulis ajukan, sebagai berikut:

1. Agar dapat meningkatkan performa agen dalam mengontrol CartPole disarankan untuk menyesuaikan parameter, menambah jumlah episode pelatihan, dan menggunakan teknik optimasi agar agen RL dapat belajar lebih baik dan mencapai kontrol optimal pada CartPole.
2. Implementasikan algoritma Deep Q-Network (DQN) dalam permasalahan yang ada di dunia nyata.

9. Daftar Pustaka

Irvansyah, Muhammad Adhitya. (2020). *IMPLEMENTASI METODE REINFORCEMENT LEARNING PADA SIMULASI PERGERAKAN ROBOT MULTI-SENDI*
